CMPE3209 Computer Organization — Proje Teslim Tutanağı

Kullanım talimatı

Bu tutanak projenin tek resmi teslim belgesidir. Her kilometre taşında (M1, M2 ve final; tümü pazartesi) ilgili adım doldurulur ve aynı dosya güncellenerek yeniden teslim edilir. Ayrıntılı görev tanımları proje yönergesindedir; bu tutanak yalnızca ne istendiğini ve nereye yazılacağını gösterir.

- Tüm sayılar birimiyle yazılır (byte, cycle, ns, GHz). Hex değerler `0x` önekiyle yazılır.
- **Varsayım** olan her değer "(varsayım)" etiketiyle işaretlenir.
- Tablolara sığmayan çıktılar (assembly listesi, objdump, CSV izi, ekran görüntüleri) Git deposuna konur. Tutanakta dosya adıyla atıf yapılır.
- Boş bırakılan zorunlu alan, o adımın puanını sıfırlar.

Öğrenci bilgileri

Alan	Değer
Ad Soyad	Furkan Akdağ
Öğrenci numarası (SID)	5001230010
E-posta	akdagfuko03@gmail.com
Seçilen görev (G1–G5)	G3 - XOR message
Seçilen ISA ve sürümü (ör. RV32IM)	RV32IM (32-bit)
Derleyici ve sürümü	Clang 17.0.0
Simülatör / araçlar	Ripes (RV32IMC); GCC C99
Git deposu adresi	github.com/vimvim01/CMPE3209-Computer-Organization

Adım 0 — Kişiye özel parametreler

Bu adım M1 ile birlikte teslim edilir. Hatalı bir parametre, ona dayanan tüm adımları geçersiz kılar; öğretim üyesi bu tabloyu M1'de kontrol eder.

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
d	SID'in son hanesi	Last digit: 0	0
D2	SID'in son iki hanesi	Last two: 10	10
S	SID'in hanelerinin toplamı	5+0+0+1+2+3+0+0+1+0	12
N	24 + (D2 mod 25)	24+(10 mod 25)	34
Cache	d'ye göre: boyut / line / mapping / line sayısı / set sayısı	d=0: direct; 8 lines; 128 B / 16 B per line	128 B 16 B 8 lines

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
Write policy	d çift → write-back, d tek → write-through	d=0 (even)	write-back
k	(S mod 7) + 3	(12 mod 7)+3	8
Interrupt türü	S çift → Timer, S tek → I/O	S=12 (even)	Timer
f	1.0 + 0.1 × d GHz (varsayım)	1.0 + 0.1 x 0	1.0 GHz (assumed)
T1 / T2	T1 = 1 cycle; T2 = 20 + 5 × (d mod 4) cycle (varsayım)	T1=1; T2=20+5 x 0	1 / 20 cycles (assumed)
Nc	2 + (S mod 7) (varsayım)	2+(12 mod 7)	7 (assumed)
Görev parametresi	Seçilen göreve göre (T, P, K, yön veya R)	1230010 mod 256	186 0xBA

Adres alanları (ISA'nın adres genişliği s + w bit; 64-bit ISA'da düşük 32 bit kullanılır):

Alan	Bit sayısı	Hesap
Word (w)	4 bits	log2(16 B)
Line (r) veya set (d)	3 bits	log2(8 lines)
Tag (s − r veya s − d)	25 bits	32 - 4 - 3
Toplam	32 bits	25 + 3 + 4

Adım 1 — Görev seçimi ve C programı (M1: 2 Kasım, Pazartesi)

1.1 Görev seçimi (yalnızca birini işaretleyin):

- ☐ G1 Biyomedikal sinyal alarmı
- ☐ G2 Sınav notu dağılımı
- ☒ G3 Mesaj şifreleme ve bütünlük
- ☐ G4 Sıralama ve medyan
- ☐ G5 Run-length sıkıştırma

1.2 Çözüm tasarımı (3–5 cümle: hangi algoritma, hangi fonksiyonlar, hangi veri yerleşimi ve neden):

An SID-seeded LCG generates 34 uppercase values in the global data[N] array. generate_and_encrypt uses xor_transform to encrypt each value in place with K = 0xBA. calculate_checksum rotates and adds the encrypted values to form a 32-bit checksum. decrypt_and_verify reverses XOR and compares each value with the regenerated LCG sequence; in-place storage avoids a second array.

1.3 C kaynak kodu. Satır numaralı kod aşağıdaki bloka yapıştırılır (25–60 satır):

59 satirlik numarali C99 kaynak kodu bu bolumun sonraki sayfasindadir.

Adim 1 - 1.3 C Kaynak Kodu (59 Satir)

Resmi tutanagin 1.3 bolumu - ek degil, dogrudan C kodu alani devamidir.

g3_message_encryption.c | C99 | RV32IM | SID = 1230010

```c
#define N 34
const int SID = 1230010;
int data[N];
unsigned int checksum = 0;
int decrypt_ok = 0;
int xor_transform(int value, int key)
{
    return value ^ key;
}
void generate_and_encrypt(int key)
{
    unsigned int state = (unsigned int)SID;
    int i = 0;
    while (i < N)
    {
        state = 1664525u * state + 1013904223u;
        data[i] = (int)(((state >> 16) % 26u) + 65u);
        data[i] = xor_transform(data[i], key);
        i++;
    }
}
unsigned int calculate_checksum(void)
{
    unsigned int c = 0;
    int i = 0;
    while (i < N)
    {
        c = (c << 1) | (c >> 31);
        c = c + (unsigned int)data[i];
        i++;
    }
    return c;
}
int decrypt_and_verify(int key)
{
    unsigned int state = (unsigned int)SID;
    int ok = 1;
    int i = 0;
    while (i < N)
    {
        data[i] = xor_transform(data[i], key);
        state = 1664525u * state + 1013904223u;
        if (data[i] != (int)(((state >> 16) % 26u) + 65u))
            ok = 0;
        i++;
    }
    return ok;
}

int main(void)
{
    int key = SID % 256;
    if (key == 0)
        key = 0xA5;
    generate_and_encrypt(key);
    checksum = calculate_checksum();
    decrypt_ok = decrypt_and_verify(key);
    return 0;
}
```

Program SID = 1230010 (last nine digits of Student ID 5001230010, per original instructions).
ISA rationale: RV32IM offers fixed 32-bit encodings and supports the LCG multiply via M extension.
C source code unchanged from the initial two-page G3 proposal.

Adim 1 - Form devam sayfasi

1.4 Zorunlu yapı eşlemesi. Her madde için satır numarası ve tek cümlelik açıklama:

#	Zorunlu yapı	Satır no	Açıklama
Y1	N elemanlı global int dizisi, en az iki geçiş	3; 14-20; 26-31; 39-46	data[N], 3 array passes
Y2	Koşullu dallanma	43; 53	Two if conditions

Zorunlu yapilar Y3-Y7, resmi formun sonraki sayfasinda devam eder.

#	Zorunlu yapı	Satır no	Açıklama
Y3	Parametreli, değer döndüren fonksiyon çağrısı	6-9; 18; 41	xor_transform call + return
Y4	Toplama/çıkarma, çarpma veya shift, bit işlemi	8; 16-18; 28-29; 42	LCG, shift, XOR, rotate
Y5	Dizi elemanına store	17-18; 41	Array value is updated
Y6	const int SID ve hesapta kullanımı	2; 12; 36; 52	SID used for LCG and key
Y7	Sonucun tutulduğu global değişken(ler)	4-5; 56-57	Global results: checksum, ok

1.5 Program çıktıları. Simülatörde çalıştırıldıktan sonra bellekten okunan değerler:

Global değişken	Adres (hex)	Değer	Ekran görüntüsü dosyası
checksum	0x00012430	23899846	M1_Ripes_Memory_Results.png
decrypt_ok	0x00012434	1	M1_Ripes_Memory_Results.png
data[0] (after decrypt)	0x00012438	84	M1_Ripes_Memory_Results.png

1.6 İzlenecek kernel (12–20 dinamik instruction; en az bir load, store, ALU, koşullu branch ve call/return içermeli):

Alan	Değer
C satır aralığı	26–32; 56–57
Başlangıç ve bitiş adresi (hex)	0x0001128C to 0x00011410
Dinamik instruction sayısı	14 (verified ELF)
Hangi döngü iterasyonu (ör. i = 3)	i = 34 (after i = 33)

Öğretim üyesi (M1): ☐ Onaylandı ☐ Düzeltme istendi ☐ Reddedildi — Tarih: _____ Not: _____

M1 14-step kernel proof: M2 Appendix E (original ELF; 0x1128C -> 0x11410).

Adım 2 — Assembly ve machine code (M2: 9 Kasım, Pazartesi)

2a. C → assembly eşlemesi (-O0; listenin tamamı depoda asm_O0.s):

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
12	state = SID	lui; addi; sw	0x10048-0x10050
13	i = 0	li; sw	0x10054-0x10058
14	while (i < 34)	lw; li; bge; j; j	0x10060-0x10070
16	LCG state update	lw,lui,addi,mul,add,sw	0x10074-0x10090
17	make data[i]	srli,mulhu,mul,sub,sw	0x10094-0x100D0
18-19	XOR, i++, back-edge	lw,slli,jal,sw,addi,j	0x100D4-0x1010C

Bölüm	Başlangıç adresi (hex)	Boyut (byte)	İçerik
.text	0x00010000	824	Code
.data / .bss	0x1233C; 0x12344	8 + 136	Globals

Von Neumann ilkesi bu adreslerde nasıl görünüyor? (1–2 cümle)

Instructions (.text) and global data (.rodata/.sbss/.bss) occupy one ELF address space.
PC fetches code; lw/sw address data. Separate sections do not imply separate physical RAM.

2b. -O0 ve -O1 karşılaştırması (kernel için):

Ölçü	-O0	-O1	Farkın nedeni
Kernel'deki instruction sayısı	36	13	Streamlined loop
Load sayısı	9	1	Register allocation
Store sayısı	7	0	No stack spills
Kullanılan register sayısı	6	5	Register reuse

2c. Elle machine code (en az 10 instruction; x86-64 için 8). Kullanılan tüm formatlar kapsanmalı.

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?
1	0x1008C	add a0,a0,a1	R	App. C	0x00B50533	33 05 B5 00	Yes
2	0x1005C	addi a2,a2,-1	I	App. C	0xFFF60613	13 06 F6 FF	Yes
3	0x10044	sw a0,-12(s0)	S	App. C	0xFEA42A23	23 2A A4 FE	Yes
4	0x10068	bge a0,a1,+8	B	App. C	0x00B55463	63 54 B5 00	Yes
5	0x10048	lui a0,300	U	App. C	0x0012C537	37 C5 12 00	Yes
6	0x100E8	jal ra,-232	J	App. C	0xF19FF0EF	EF F0 9F F1	Yes
7	0x10074	lw a0,-16(s0)	I	App. C	0xFF042503	03 25 04 FF	Yes
8	0x10080	mul a0,a0,a1	R	App. C	0x02B50533	33 05 B5 02	Yes
9	0x10020	xor a0,a0,a1	R	App. C	0x00B54533	33 45 B5 00	Yes
10	0x1004C	addi a0,a0,1210	I	App. C	0x4BA50513	13 05 A5 4B	Yes

Zorunlu instruction'ların tablodaki sıra numarası:

- ☒ SID'i yükleyen instruction(lar): # 5,10
- ☒ Koşullu branch: # 4
- ☒ Load: # 7
- ☒ Store: # 3
- ☒ Fonksiyon çağrısı: # 6

2c binary bit fields: M2 Appendix C. ELF / simulator evidence: M2 Appendix D.

SID'in yüklenmesi. Sabitin neden tek instruction'a sığmadığı, nasıl bölündüğü ve gerekiyorsa +1 düzeltmenin sign extension ile açıklaması:

SID = 1230010 = (300 << 12) + 1210. lui a0,300 then addi a0,a0,1210.
ADDI immediate is signed 12-bit; 1210 fits, so no upper-part +1 correction is required.

Branch offset hesabı. Branch'in adresi, hedef adres ve offset'in adım adım hesabı:

BGE at 0x10068, target 0x10070: offset = 0x10070 - 0x10068 = +8 byte.
B-type imm fields: 0 | 000000 | 0100 | 0; encoding 0x00B55463.

Adım 3 — CPU, interrupt ve bus (final paketiyle: 16 Kasım, Pazartesi)

Model varsayımları (okudum ve uyguladım):

- ☐ Pipeline yok, her an tek instruction yürütülür.
- ☐ PC, IR, MAR ve MBR Fig. 3.2'deki gibi kavramsal olarak vardır.
- ☐ Her bellek erişimi bir bus işlemidir; cache bu adımda yok sayılır.
- ☐ Data bus genişliği = ____ bit.

Modelin gerçek donanımdan ayrıldığı yerler (3–5 cümle):

3a. Instruction cycle izleme tablosu. Her dinamik instruction bir satırdır. Bir instruction birden çok bellek erişimi yapıyorsa MAR, MBR ve bus hücrelerine sırayla yazılır. Kategori: PM = processor–memory, PIO = processor–I/O, DP = data processing, C = control.

#	PC (önce → sonra)	Assembly	IR (hex)	Fig. 3.6 durumları	MAR	MBR	ALU işlemi	Değişen register	Bus (adres / data / control)	Kategori
1										
2										
3										
4										
5										
6										
7										
8										
9										
10										
11										
12										

#	PC (önce → sonra)	Assembly	IR (hex)	Fig. 3.6 durumları	MAR	MBR	ALU işlemi	Değişen register	Bus (adres / data / control)	Kategori
13										
14										
15										
16										
17										
18										
19										
20										

Kernel toplamları	Sayı
Instruction fetch	
Operand fetch (of)	
Operand store (os)	
PC'nin sıra dışı değiştiği instruction	

3b. Fig. 3.4 makinesine aktarım. Kernel'in tek bir dizi elemanını güncelleyen parçası, opcode 1, 2, 3, 5 ve 7 ile yeniden yazılır ve Fig. 3.5 formatında izlenir:

Adım	Fetch / execute	PC	IR (hex)	AC	Değişen bellek hücresi
1					
2					
3					
4					
5					
6					
7					
8					

Ölçü	Seçilen ISA	Fig. 3.4 makinesi
Aynı iş için instruction cycle sayısı		

Tek accumulator'ın getirdiği sınırlama (PDP-11 ADD B,A ile karşılaştırarak):

IAS register'ları ve seçilen ISA:

IAS register'ı	IAS'taki görevi	Seçilen ISA'daki karşılığı	Programcıya görünür mü?
MBR			
MAR			
IR			
IBR			
PC			
AC			
MQ			

3c. Interrupt senaryosu (k = ____, tür = ____):

Soru	Cevap
Interrupt hangi instruction'ın execute'u sırasında geliyor? (adres, assembly)	
Ne zaman tanınır ve bu instruction neden yarıda kesilmez? (Fig. 3.9)	
Kaydedilen PC değeri (hex)	
Kaydedilen diğer durum bilgisi	
Seçilen ISA'daki gerçek mekanizma (register adları, vektör)	
Kaynak (ISA referansı, bölüm, sayfa)	
Handler dönüşünde ilk çalışan instruction'ın adresi	

Fonksiyon çağrısı (Y3) ve interrupt karşılaştırması:

Ölçüt	Fonksiyon çağrısı	Interrupt
Akışı kim başlatır?		
Dönüş adresini kim kaydeder, nereye?		
User program bundan haberdar mı?		

İkinci (daha yüksek öncelikli) interrupt. Aynı olay dizisini iki yaklaşımla zaman çizelgesi olarak doldurun (Fig. 3.13 ve 3.14):

Zaman	Sequential (disable)	Nested (priority)
t_0: 1. interrupt gelir		
t_1: 2. interrupt gelir		
t_2		
t_3		
t_4: user program'a dönüş		

3d. Bus ve interconnection.

Bus işlemi türü (kernel, cache yok)	Sayı	Aktif control line'lar
Instruction fetch (read)		
Data read		
Data write		
Toplam		

Hesap	Değer	Adımlar
Address bus genişliği ve maksimum adreslenebilir bellek		
Data bus 16 bit olsaydı kernel'in bus cycle sayısı		
Mevcut / 16-bit oranı		

PCIe paket verimi (N × 4 = ____ byte'lık tek Memory Write TLP; Fig. 3.25):

Alan (octet)	12-byte header, ECRC yok	16-byte header, ECRC var
STP framing (baş)		
Sequence number		
Header		
Data		
ECRC		
LCRC		
STP framing (son)		
Toplam		
Verim = data / toplam		

Alan (octet)	12-byte header, ECRC yok	16-byte header, ECRC var
128b/130b ile verim		

Sonucun bir çekirdekten PCIe cihazına yolu (Fig. 3.17 ve 3.21): her adımda bağlantı, protokol katmanı ve aktarım birimi.

Adım 4 — Cache simülasyonu (final paketiyle: 16 Kasım, Pazartesi)

4a. Data adres izi. Tüm load ve store'lar depoda `trace.csv` dosyasındadır. Biçim: `sira,RW,adres_hex,boyut,c_satiri,stack_mi`.

Ölçü	Değer
Kullanılan derleme (`-O0` / `-O1`)	
Toplam data referansı	
Load / store	
Stack erişimi sayısı	
Ana diziye erişim sayısı	

4b. İlk 5 farklı adresin alanları:

Adres (hex)	Adres (binary)	Tag	Line / Set	Word

4c. Elle simülasyon (ilk 30 referans):

#	R/W	Adres (hex)	Tag	Line / Set	Word	Hit / Miss	Yerinden çıkan blok	Bu blok daha önce cache'te miydi?
1								
2								
3								
4								
5								
6								

#	R/W	Adres (hex)	Tag	Line / Set	Word	Hit / Miss	Yerinden çıkan blok	Bu blok daha önce cache'te miydi?
7								
8								
9								
10								
11								
12								
13								
14								
15								
16								
17								
18								
19								
20								
21								
22								
23								
24								
25								
26								
27								
28								
29								
30								

Cache'in 30. referanstan sonraki anı. Line sayınız 8'den fazlaysa satır ekleyin.

Line	Set	Valid	Tag (hex)	Dirty (write-back)	LRU sırası (associative)
0					

Line	Set	Valid	Tag (hex)	Dirty (write-back)	LRU sırası (associative)
1					
2					
3					
4					
5					
6					
7					

4d. Tam simülasyon sonuçları:

Ölçü	Elle (ilk 30)	Araç (ilk 30)	Araç (tüm iz)
Referans	30	30	
Hit			
Miss			
Hit ratio H			
Ana belleğe yazma sayısı			

Kullanılan araç ve ayarları (kendi simülatörünüz varsa dosya adı):

- ☐ Elle ve araçla yapılan simülasyon ilk 30 referansta birebir örtüşüyor.

4e. Tasarım uzayı (cache boyu sabit):

Mapping (line boyu sabit = ____ B)	Hit	Miss	H
Direct			
2-way set-associative			
4-way set-associative			
Fully associative (LRU)			

Line boyu (mapping sabit = ____)	Hit	Miss	H
8 B			
16 B			

Line boyu (mapping sabit = ____)	Hit	Miss	H
32 B			
64 B			

Grafik dosyası ve eğilimlerin locality ile açıklaması (thrashing varsa çatışan adresler):

4f. Ortalama erişim süresi: $T_s = H \times T1 + (1 - H) \times (T1 + T2)$

Durum	H	T_s (cycle)	Speedup (cache'siz duruma göre)
Cache yok	0	T2 =	1
Kişiye özel cache			

4g. Kod değişikliği:

	Önce	Sonra
Değiştirilen C satır(lar)ı		
Y7 çıktısı (aynı olmalı)		
H		
T_s (cycle)		

Değişikliğin gerekçesi; H artmadıysa nedeni:

Adım 5 — Performans ve architecture/organization (final paketiyle: 16 Kasım, Pazartesi)

5a-1. Dinamik instruction dağılımı ve CPI (CPI_i değerleri yönergedeki model varsayımıdır):

Sınıf	CPI_i (varsayım)	I_i (-O0)	I_i (-O1)
ALU	1		
Çarpma	3		
Load	1		
Store	1		
Branch / jump	2		
Ic (toplam)			
Ortalama CPI			

5a-2. Süre ve MIPS rate (f = ____ GHz, τ = ____ ns):

Ölçü	Formül	-O0	-O1
T (bellek etkisi yok)	$Ic \times CPI \times \tau$		
MIPS rate	$f / (CPI \times 10^6)$		
m (instruction başına data referansı)	data ref. / Ic		
T, cache yok (k = T2)	$Ic \times [p + (m \times k)] \times \tau$		
T, cache var (k = T_s)	$Ic \times [p + (m \times k)] \times \tau$		

Karşılaştırma	Değer
Speedup -O0 → -O1 (Eq. 2.1, cache var)	
Speedup cache yok → cache var (-O0)	

MIPS rate'in iki derlemeyi karşılaştırmada neden yanıltıcı olabileceği, kendi sayılarınızla:

5a-3. Amdahl yasası (Nc = ____):

Alan	Değer
Paralelleştirilebilen kısım (C satırları)	
Bağımsızlık gerekçesi (veri bağımlılığı)	
f (yürütme süresindeki pay)	
Speedup = $1 / [(1 - f) + f / Nc]$	
$Nc \to \infty$ limiti	

5b. Architecture ve organization (en az 10 özellik):

#	Özellik	Architecture / Organization	Gerekçe (Stallings tanımı)	Değişseydi Y7 değişir miydi? Kanıt
1				
2				
3				
4				
5				
6				
7				

#	Özellik	Architecture / Organization	Gerekçe (Stallings tanımı)	Değişseydi Y7 değişir miydi? Kanıt
8				
9				
10				

Ch. 1.2'deki dört temel fonksiyon:

Fonksiyon	Kernel'den örnek instruction (adres, assembly)	Açıklama
Data processing		
Data storage		
Data movement		
Control		

Final teslim (16 Kasım Pazartesi; sözlü savunma 20 Kasım Cuma)

Teslim kontrol listesi:

- ☐ Bu tutanak, tüm adımları dolu olarak (PDF ve düzenlenebilir sürüm)
- ☐ C kaynak kodu (`main.c`) ve sonraki 4g sürümü (`main_v2.c`)
- ☐ `asm_O0.s`, `asm_O1.s` ve `objdump -d` çıktısı
- ☐ `trace.csv` ve cache simülasyon çıktıları (kendi simülatörünüz varsa kaynak kodu)
- ☐ Ekran görüntüleri: machine code doğrulaması, program çıktıları, cache durumu
- ☐ 4e grafiği
- ☐ Git deposunda M1, M2 ve final etiketleri

Kilometre taşı kaydı (öğretim üyesi doldurur):

Kilometre taşı	Son tarih	Teslim tarihi	Git etiketi	Paraf
M1 Öneri (Adım 0–1)	2 Kasım (Pzt)			
M2 Assembly ve machine code (Adım 2)	9 Kasım (Pzt)			
M3 CPU, interrupt, bus (Adım 3)	16 Kasım (Pzt, final paketiyle)			
M4 Cache ve performans (Adım 4–5)	16 Kasım (Pzt, final paketiyle)			
Final ve sözlü savunma	Teslim 16 Kasım (Pzt); savunma 20 Kasım (Cuma)			

YZ kullanım beyanı. Kullanılmadıysa ilk satıra "Kullanılmadı" yazın. Beyan edilmemiş kullanım akademik dürüstlük ihlalidir.

Araç ve sürümü	Adım	Ne için kullanıldı	Örnek prompt	Çıktıyı nasıl doğruladım
ChatGPT (GPT-6)	M1 Adım 0-1	Raporu düzenleme, eksik kısımları tamamlama	Bütün dosyaları yükledim, buna göre rapor taslağını doldur.	Rapor taslağı ile karşılaştırma
ChatGPT (GPT-6)	M1 Adım 1	Ripes rehberliği, kernel analizi	Bu aşamada ne yapmam gerektiğini bilmiyorum, beraber yapalım.	Ripes bellek resmi, ELF disassembly
ChatGPT (GPT-6)	M2 / 2a-2c	Assembly tutoring; report drafting	Review my RISC-V encoding + ELF.	Manual work vs LLVM; Ripes memory checks

Öğrenci beyanı. Bu tutanaktaki C kodunu, elle machine code kodlamasını, instruction cycle izleme tablosunu ve elle cache simülasyonunu kendim yaptım. Başka bir öğrencinin çalışmasını kopyalamadım ve kendi çalışmamı paylaşmadım. YZ kullanımımı eksiksiz beyan ettim. Sözlü savunmada bu çalışmayı açıklayamamamın puanımı etkileyeceğini biliyorum.

Ad Soyad	Öğrenci numarası	Tarih	İmza
Furkan Akdağ	5001230010		

Değerlendirme (öğretim üyesi doldurur)

Bileşen	Azami	Puan	Not
A1 Öneri ve parametre hesabı (Adım 0–1)	5		
A2 Assembly ve machine code (Adım 2)	15		
A3 CPU, interrupt, bus (Adım 3)	25		
A4 Cache (Adım 4)	25		
A5 Performans ve architecture/organization (Adım 5)	10		
Tutanak ve rapor kalitesi	5		
Sözlü savunma	15		
Geç teslim kesintisi	–		
Toplam	**100**		

Sözlü savunma kaydı:

Madde	Sorulan	Sonuç (başarılı / kısmen / başarısız)	Puan
1. Canlı encode (`-O1` listesinden)			
2. Adres alanları ve hit/miss			
3. Interrupt k+1'de gelseydi			
4. Tasarım kararı savunması			

☐ 1. ve 2. maddelerin ikisi de başarısız: A2–A4 puanları 0.5 ile çarpıldı.

Öğretim üyesi	Tarih	İmza

M2 Appendix A - C to Assembly / ELF

Based on g3_message_encryption.c; RV32IM / ILP32; linked -O0 ELF

Build / link parameters

clang-17 --target=riscv32 -march=rv32im -mabi=ilp32 -O0 -c g3_message_encryption.c -o g3_O0.o
ld.lld-17 -m elf32lriscv -e main -Ttext=0x10000 -o g3_O0.elf g3_O0.o
Files: m2_evidence/build_O0.txt, sections_O0.txt, symbols_O0.txt, disassembly_O0.txt
M2 page 4 addresses come from the new Ubuntu ELF, not the earlier M1 Windows ELF.

Memory sections and global symbols

Section / symbol	Start	Size	Description
.text	0x10000	824 B	Executable code
.rodata / SID	0x11338	4 B	Read-only student-derived SID
.sbss / checksum	0x1233C	4 B	Zero-initialized result
.sbss / decrypt_ok	0x12340	4 B	Verification flag
.bss / data[34]	0x12344	136 B	34 x 4-byte integers

Detailed C-line mapping (generate_and_encrypt)

C line	Purpose	ELF address range	Main ops / blocks
10-11	Prologue; save key	0x10034-0x10044	addi, sw
12	state = SID	0x10048-0x10050	lui, addi, sw
13	i = 0	0x10054-0x10058	li, sw
14	while (i < 34)	0x10060-0x10070	lw, li, bge, j, j
16	LCG state update	0x10074-0x10090	lw, lui, addi, mul, add, sw
17	A-Z generation; data[i]	0x10094-0x100D0	srli, mulhu, mul, sub, slli, sw
18	XOR function and store	0x100D4-0x100FC	lw, slli, jal, sw
19	i++ ; jump to condition	0x10100-0x1010C	lw, addi, sw, j
21	Epilogue	0x10110-0x1011C	lw, addi, ret

Von Neumann interpretation

ELF sections .text and .rodata/.sbss/.bss are different logical sections inside one program address space.

The program counter fetches executable instructions and lw/sw access data addresses. The ELF alone does not prove the physical memory implementation.

M2 Appendix B - Kernel comparison

M1 trace: checksum path (C 26-32, 56-57). 2b: static enclosing function, calculate_checksum().

2b: Comparable function, calculate_checksum() (-O0 vs -O1)

Metric	-O0	-O1	Basis / explanation
Code size	144 B	52 B	From user-provided ELF symbol tables
Static instructions	36	13	Counted in ELF disassembly; 4-byte ISA
Load (lw)	9	1	Stack spills removed; in-register loop
Store (sw)	7	0	No stack temporary stores in -O1
Distinct GPRs	6	5	-O1 keeps live values in registers

Scope and interpretation

All values above were checked against the student's actual Ubuntu Clang/LLD 17.0.6 ELF disassembly (8 Nov 2026). -O0: 0x10120-0x101AC; -O1: 0x1006C-0x1009C.

M1 section 1.6 selected a 14-dynamic-instruction trace from the last checksum iteration and nearby call/return. 2b uses the complete calculate_checksum() function containing that code, so the 36 and 13 counts are static whole-function counts, NOT the 14-step dynamic trace.

For the regular, taken iteration only (including loop condition and back-edge, excluding entry and exit), -O0 executes 22 instructions including 6 loads and 3 stores; -O1 executes 8 instructions including 1 load and 0 stores.

The previously measured 59 vs 25 comparison concerns generate_and_encrypt(), not the M1 checksum kernel, and is shown separately for context.

Supplementary: generate_and_encrypt(), not the M1 kernel

Metric	-O0	-O1
Static instructions	59	25
Load	12	0
Store	10	1
Distinct GPRs	6	10

Ubuntu ELF ranges used for verified counts

```
llvm-objdump-17 -d --no-show-raw-insn --start-address=0x10120 --stop-address=0x101B0 g3_O0.elf
```

```
llvm-objdump-17 -d --no-show-raw-insn --start-address=0x1006C --stop-address=0x100A0 g3_O1.elf
```

M2 Appendix C - Manual instruction encoding

All six RV32I formats are covered; multiply uses the M extension (RV32IM).

Bit-field notation: R=f7|rs2|rs1|f3|rd|op; I=imm12|rs1|f3|rd|op; S=immHi7|rs2|rs1|f3|immLo5|op.
B=imm12|imm10:5|rs2|rs1|f3|imm4:1|imm11|op; U=imm20|rd|op; J=imm20|imm10:1|imm11|imm19:12|rd|op.

01 add a0,a0,a1 [R; 0x1008C; 0x00B50533; LE 33 05 B5 00]

f7=0000000 | rs2=01011 | rs1=01010 | f3=000 | rd=01010 | op=0110011
32-bit binary: 00000000101101010000010100110011 | objdump: Yes

02 addi a2,a2,-1 [I; 0x1005C; 0xFFF60613; LE 13 06 F6 FF]

imm12=111111111111 | rs1=01100 | f3=000 | rd=01100 | op=0010011
32-bit binary: 11111111111101100000011000010011 | objdump: Yes

03 sw a0,-12(s0) [S; 0x10044; 0xFEA42A23; LE 23 2A A4 FE]

imm11:5=1111111 | rs2=01010 | rs1=01000 | f3=010 | imm4:0=10100 | op=0100011
32-bit binary: 11111110101001000010101000100011 | objdump: Yes

04 bge a0,a1,+8 [B; 0x10068; 0x00B55463; LE 63 54 B5 00]

imm12=0 | imm10:5=000000 | rs2=01011 | rs1=01010 | f3=101 | imm4:1=0100 | imm11=0 | op=1100011
32-bit binary: 00000000101101010101010001100011 | objdump: Yes

05 lui a0,300 [U; 0x10048; 0x0012C537; LE 37 C5 12 00]

imm20=00000000000100101100 | rd=01010 | op=0110111
32-bit binary: 00000000000100101100010100110111 | objdump: Yes

06 jal ra,-232 [J; 0x100E8; 0xF19FF0EF; LE EF F0 9F F1]

imm20=1 | imm10:1=1110001100 | imm11=1 | imm19:12=11111111 | rd=00001 | op=1101111
32-bit binary: 11110001100111111111000011101111 | objdump: Yes

07 lw a0,-16(s0) [I; 0x10074; 0xFF042503; LE 03 25 04 FF]

imm12=111111110000 | rs1=01000 | f3=010 | rd=01010 | op=0000011
32-bit binary: 11111111000001000010010100000011 | objdump: Yes

08 mul a0,a0,a1 [R; 0x10080; 0x02B50533; LE 33 05 B5 02]

f7=0000001 | rs2=01011 | rs1=01010 | f3=000 | rd=01010 | op=0110011
32-bit binary: 00000010101101010000010100110011 | objdump: Yes

09 xor a0,a0,a1 [R; 0x10020; 0x00B54533; LE 33 45 B5 00]

f7=0000000 | rs2=01011 | rs1=01010 | f3=100 | rd=01010 | op=0110011
32-bit binary: 00000000101101010100010100110011 | objdump: Yes

10 addi a0,a0,1210 [I; 0x1004C; 0x4BA50513; LE 13 05 A5 4B]

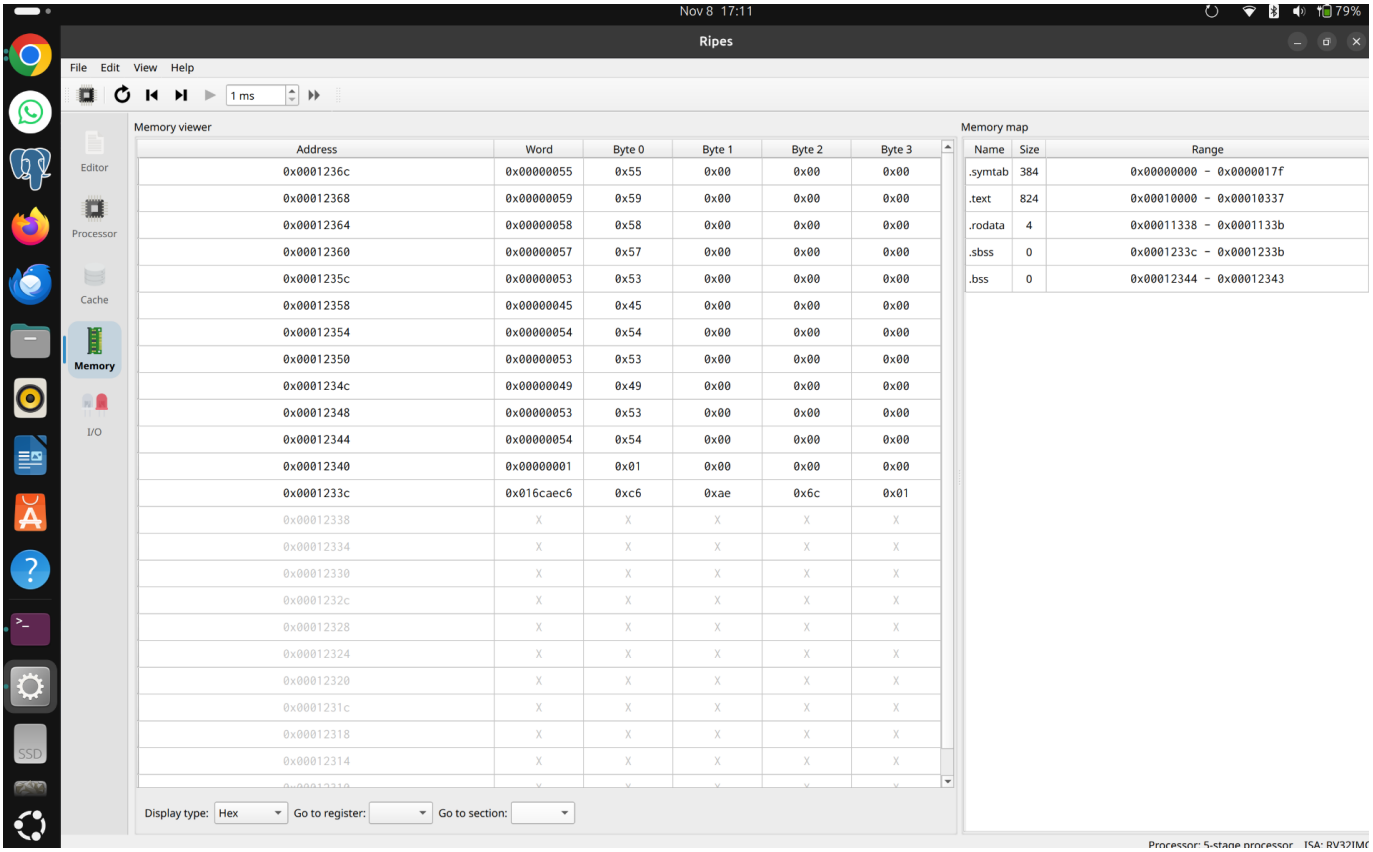
imm12=010010111010 | rs1=01010 | f3=000 | rd=01010 | op=0010011
32-bit binary: 01001011101001010000010100010011 | objdump: Yes

M2 Appendix D - Ripes verification

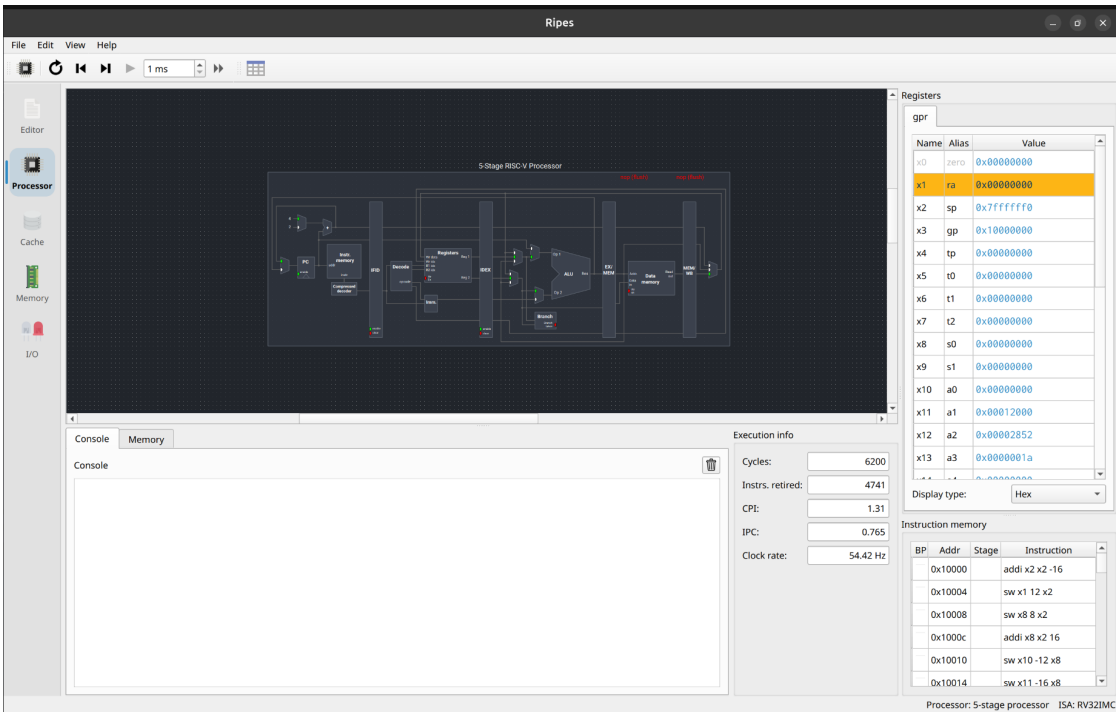
5-stage RV32IMC processor selected; RV32IM instruction subset used by the program.

Program output / memory verification (after execution)

checksum = 0x016CAEC6; decrypt_ok = 1; data[0] = 0x54 (84, ASCII T)



Processor execution snapshot



Ripes: 6200 cycles; 4741 retired instructions; CPI 1.31 (observed full -O0 run).

ISA note: compiled RV32IM (no C); Ripes RV32IMC can execute its 32-bit instructions.

M2 Appendix E - Approved M1 Kernel: objdump Evidence

Original M1 ELF: g3_message_encryption_rv32im.elf (not the newer Ubuntu g3_O0.elf)
Approved dynamic trace: start PC 0x1128C; end PC 0x11410; 14 retired instructions.
Execution jumps to nonadjacent PCs; this is a dynamic trace, not a linear .text address interval.

#	PC	Word (hex)	Assembly / operation	Next PC	Type / explanation
1	0x1128C	0xFADFF06F	j 0x11238	0x11238	Jump / loop back
2	0x11238	0xFF042583	lw a1,-16(s0)	0x1123C	Load i = 34
3	0x1123C	0x02100513	li a0,33	0x11240	ALU / compare limit
4	0x11240	0x04B54863	blt a0,a1,0x11290	0x11290	Branch taken: 33 < 34
5	0x11290	0xFF442503	lw a0,-12(s0)	0x11294	Load checksum
6	0x11294	0x00C12083	lw ra,12(sp)	0x11298	Restore return addr
7	0x11298	0x00812403	lw s0,8(sp)	0x1129C	Restore frame ptr
8	0x1129C	0x01010113	addi sp,sp,16	0x112A0	Restore stack
9	0x112A0	0x00008067	ret	0x11400	Return to main
10	0x11400	0x000125B7	lui a1,0x12	0x11404	Load global address high
11	0x11404	0x42A5A823	sw a0,0x430(a1)	0x11408	Store checksum
12	0x11408	0xFF042503	lw a0,-16(s0)	0x1140C	Load XOR key
13	0x1140C	0x00000097	auipc ra,0	0x11410	Call target base
14	0x11410	0xE98080E7	jalr -0x168(ra)	0x112A4	Call decrypt_and_verify

Key checks

1) 0x11240 BLT branches to 0x11290 because the loop counter is i=34 (>33).
2) 0x112A0 RET returns to main at 0x11400, where checksum is stored.
3) 0x1140C AUIPC + 0x11410 JALR call decrypt_and_verify at 0x112A4.
4) Original ELF: readelf -A -> rv32i2p1_m2p0_zmmul1p0, with NO C extension.
5) M2 Ubuntu builds use -march=rv32im -mabi=ilp32. Ripes RV32IMC accepts RV32IM.

Git evidence: M2_Hoca_Kanitlari/M1_approved_kernel_14_instructions_objdump.txt